

TECHNICAL DOCUMENTATION

雪 (Yuki) Bot

完整技术栈文档

面向初学者的技术解析

从 Telegram Bot 到 3D VRM 渲染、记忆系统到 LoRA 训练的全栈图解

目录

1 项目是什么 — 整体概述

2 Telegram Bot 核心

tg_daemon / handle_reply / lt_daemon / claude_monitor

3 AI / LLM 接入层

Claude CLI、多后端统一客户端、提示词工程

4 记忆系统

SQLite 数据库、向量检索、热度衰减、Dream 整合

5 管理面板 (Web UI)

FastAPI 后端、前端 Bundle、REST API

6 3D VRM 模型 & 渲染

Three.js、@pixiv/three-vrm、Idle 动画、无头渲染

7 桌面应用 (Electron)

8 图像生成 & LoRA 训练

Stable Diffusion、kohya-ss sd-scripts、训练流水线

9 语音合成 (GPT-SoVITS)

10 技术栈全览 & 数据流

雪 Bot 是一个运行在本地 Windows PC 上的 AI 陪伴系统。它通过 Telegram 与用户聊天，有自己的长期记忆、定时主动发消息、语音回复、3D 模型显示和生图能力。

对初学者的比喻：把它想象成一个住在你电脑里的虚拟角色。她记得你们聊过的事，会自己想起来给你发消息，还有 3D 形象可以看。

项目目录结构

```
F:\bot\ ← 主代码仓库 |— core/ ← Bot 运行时 (Python) |— memory/ ← 记忆系统 + 后台服务 | |— admin/ ← Web 管理面板 |— desktop/ ← 桌面 Electron 应用 |— lora/ ← LoRA 训练流水线 |— prompts/ ← AI 任务提示词 (7个) |— assets/ ← 模型权重、语音资源 |— data/ ← 运行时数据库和日志 |— character.txt ← 角色人设 (唯一需要改的文件)
```

五大核心能力

能力	做什么	用到的技术
💬 聊天回复	接收 Telegram 消息，用 AI 生成回复	Python、Claude API、Telegram Bot API
🧠 长期记忆	记住聊过的事、情绪、约定	SQLite、向量数据库、sentence-transformers
🕒 主动消息	定时自己发消息（不等对方先说）	Claude CLI、文件信号机制
🗣️ 语音回复	将文字转成角色声音发送	GPT-SoVITS
🖼️ 图像 / 3D	3D 模型展示、AI 生图	Three.js、Stable Diffusion、LoRA

Bot 核心由四个 Python 脚本组成，它们**并行运行**，通过写/读本地 JSON 文件来传递任务，互不干扰。

为什么这样设计？ 任何一个进程崩溃都不影响其他两个。tg_daemon 崩了不影响主动消息，lt_daemon 崩了不影响回复聊天。

消息回复流程

薰发来消息 | ▼ `tg_daemon.py` ← 每 30 秒轮询 Telegram API | 发现新消息 | 写入 `data/pending_reply.json` | 直接启动 `handle_reply.py` ▼ `handle_reply.py` | — 读记忆 (`lt_interface.py`) | — 调 LLM API → 生成回复文字 | — 调 GPT-SoVITS → 生成语音 (可选) | — 发 Telegram 消息 | — 写回忆 (存对话记录) | ▼ 删除 `pending_reply.json` → 完成

Life Tick — 主动消息流程

"Life Tick" 是雪自己决定要不要给你发消息的机制，每 15~45 分钟触发一次。

`lt_daemon.py` ← 计时器 (每 15~45 分钟) | 到时间后写入 `data/pending_tick.json` ▼ `claude_monitor.py` ← 监视 `data/` 目录的文件变化 | 发现 `pending_tick.json` | 执行 `claude -p lifetick_prompt.txt` ▼ Claude 读取 `character.txt` + 记忆 | — 决策: 现在要不要发消息? | — 如果发: 生成内容并发 Telegram | — 写 `data/next_tick.json` (下次时间)

四个脚本的职责

脚本	类型	职责
<code>tg_daemon.py</code>	常驻进程	轮询 Telegram，发现消息就写信号文件
<code>handle_reply.py</code>	回复引擎	读消息→查记忆→调 AI→发回复→存记录
<code>lt_daemon.py</code>	常驻进程	纯计时器，到点写信号文件
<code>claude_monitor.py</code>	常驻进程	监控文件变化，调度所有 AI 任务

信号文件机制：进程之间不通过网络或共享内存通信，而是通过创建/删除 JSON 文件来传递"任务"。这种设计极其简单可靠，任何文本编辑器都能看到系统当前在做什么。

什么是 LLM?

LLM (Large Language Model, 大语言模型) 是让 Bot 能"理解"和"生成"文字的核心 AI。雪 Bot 支持多种 LLM 后端, 可以在配置文件里切换。

支持的 AI 后端

后端	代表模型	特点	需要什么
Claude	claude-sonnet-4-6	推荐, JSON 输出最稳定	Claude CLI 登录
OpenAI	gpt-4o	通用, 效果好	API Key (付费)
DeepSeek	deepseek-chat	中文能力强, 价格低	API Key
Gemini	gemini-2.0-flash	速度快	API Key
Ollama	llama3、qwen2.5	完全本地, 无需联网	本地 GPU
Custom	任意模型	兼容 OpenAI API 格式的任何服务	自定义 URL

Claude CLI 的工作方式

雪 Bot 的主要 AI 调用方式是 `claude -p 提示词文件.txt`, 这是 Anthropic 官方的命令行工具:

```
# 不是直接调 API, 而是调 claude 命令行工具 claude -p lifetick_prompt.txt # claude 会: 1. 读取 lifetick_prompt.txt 的内容作为系统提示 2. 让 AI 执行里面描述的任务 (读文件、写文件、调工具) 3. 返回结果
```

提示词工程 — 7 个任务文件

Bot 的"大脑逻辑"不是写在代码里的, 而是写在这 7 个提示词文件里:

文件	什么时候用	做什么
<code>reply_prompt.txt</code>	每条消息	完整回复流程: 读记忆→生成→发送→存储
<code>lifetick_prompt.txt</code>	每次 Tick	决策是否主动发消息, 写生活日志
<code>wakeup_prompt.txt</code>	每天早上	处理睡觉期间积压的未读消息

<code>lt_selfcheck_prompt.txt</code>	每次回复后	检查计时系统是否正常运行
<code>recovery_prompt.txt</code>	开机时	补全离线期间缺失的记录，恢复进程
<code>archive_prompt.txt</code>	手动触发	整理归档对话
<code>diary_prompt.txt</code>	手动触发	生成角色视角的日记

设计理念：把 AI 逻辑放在提示词文件里而不是代码里，意味着你可以在不改一行 Python 代码的情况下，完全改变 Bot 的行为——只需编辑文本文件。

为什么需要记忆系统？

普通 AI 对话是无状态的——每次对话结束，AI 就"忘了"所有内容。雪 Bot 通过专门的记忆系统，让她能记住你们聊过的事，就像真实的人际关系。

数据库结构（SQLite）

所有数据存在一个 SQLite 文件里（`data/memory.db`），SQLite 就是一个存在单个文件里的轻量数据库，不需要单独安装服务器：

数据表	存什么	示例
<code>memories</code>	长期记忆片段	"薰不喜欢甜的"、"答应见面做番茄炒蛋"
<code>conversations</code>	所有对话记录	每条消息的内容、时间、情绪
<code>life_logs</code>	生活日志	每次 Tick 的活动、心情、是否发消息
<code>calendar_pages</code>	日程记忆	重要事件、约定
<code>dream_logs</code>	记忆整合记录	Dream 系统的运行历史
<code>config</code>	系统配置	各种调节参数

向量检索 — 怎么找到"相关"记忆

存了几百条记忆后，如何快速找到"和当前话题最相关"的记忆？答案是**向量检索**：

- 1 文字 → 数字向量**：每条记忆存入时，用 `BAAI/bge-small-zh-v1.5` 模型把文字转成 512 个数字组成的向量（代表语义）
- 2 查询时同样转换**：当前对话内容也转成向量
- 3 计算余弦相似度**：向量越接近，语义越相似，返回最相关的 15 条记忆

为什么用中文专用模型？ BAAI/bge-small-zh-v1.5 是专门针对中文优化的嵌入模型，只有 90MB，能在 CPU 上快速运行，不需要 GPU。

热度衰减系统

并非所有记忆都一直重要。就像人类记忆一样，雪 Bot 的记忆有"热度"概念：

类型	热度半衰期	说明
普通记忆	3 天	不被访问的话，3 天后热度降一半
重要记忆	7 天	重要性 ≥ 7 分的记忆衰减更慢
永久记忆	不衰减	被标记为 permanent 的核心记忆
被访问	热度延长	每次被检索到，热度会回升

Dream 记忆整合

当碎片记忆积累超过 30 条，Dream 系统会被触发：让 AI 阅读所有碎片，合并、归纳成结构化的长期记忆——类比人类睡眠时大脑整理记忆。

管理面板是什么？

管理面板是一个跑在本地的网页应用，地址是 `http://127.0.0.1:8765`。用浏览器打开就能看到 Bot 的所有数据，不需要懂命令行。

后端：FastAPI

FastAPI 是一个 Python Web 框架，负责处理前端的请求，读写数据库：

浏览器 (`index.html`) | HTTP 请求 ▼ **FastAPI** `server.py` :8765 |— GET `/memories` → 查询记忆列表 |— POST `/memories` → 添加记忆 |— GET `/memories/search` → 语义搜索 |— GET `/status` → 今日状态 |— GET `/conversations` → 对话历史 |— GET `/model/vrm` → 提供 VRM 3D 模型文件 |— GET `/vrm-viewer` → VRM 查看器页面

VRM 查看器（新功能）

管理面板首页内嵌了一个 3D 模型查看器——你可以直接在浏览器里旋转查看雪的 3D 形象：

实现方式：一个独立的 HTML 页面（`vrm_viewer.html`）通过 `iframe` 嵌入到管理面板首页，用 Three.js 加载 VRM 文件并渲染，带有呼吸动画和随机眨眼效果。

前端技术

FastAPI

Uvicorn (ASGI服务器)

Three.js

@pixiv/three-vrm

SQLite

Pydantic (数据验证)

什么是 VRM?

VRM 是一种专门为虚拟角色设计的 3D 模型格式（基于 glTF），广泛用于 VTuber 和虚拟形象。它包含：3D 网格、骨骼绑定、材质、表情、物理弹簧等。

Three.js — 在浏览器里画 3D

Three.js 是一个 JavaScript 库，让你在浏览器里用 WebGL 渲染 3D 场景，不需要安装任何插件：

概念	解释	类比
Scene	3D 场景容器	舞台
Camera	观察视角	摄像机
Renderer	WebGL 渲染器	把场景画到屏幕上
Mesh	3D 物体	演员
Light	光源	打光
OrbitControls	鼠标控制旋转	导演转镜头

Idle 动画系统

让 3D 模型自然"活着"——不是播放预录的动画，而是用数学公式实时计算：

```
// 呼吸：多个不同频率的正弦波叠加
const breath = sin(t × 0.78) × 0.013 // 主呼吸频率
               + sin(t × 1.56) × 0.003 // 二次谐波
               + sin(t × 0.31) × 0.005 // 低频起伏

// 应用到骨骼：胸腔随呼吸起伏
chest.rotation.x = breath × 0.55

// 随机眨眼：状态机
wait → closing (70ms) → opening (120ms) → wait (3~7秒后再眨)
```

无头渲染 (vrm_render.py)

用于生成 LoRA 训练数据时，需要在没有显示器的情况下截图。解决方案：

- 2 加载包含 Three.js + VRM 渲染代码的 HTML 页面
- 3 等待渲染完成，读取 `canvas.toDataURL()` 获取 base64 图片
- 4 解码 base64，保存为 PNG 文件

巧妙之处：用浏览器来渲染 3D，意味着不需要安装 OpenGL、DirectX 等复杂图形库——Chromium 自带 WebGL 支持，跨平台。

Electron 是什么？

Electron 让你用 HTML/CSS/JavaScript（网页技术）来写桌面应用程序。VS Code、Discord 都用 Electron 构建。

Electron — 主进程 (main.js) ← Node.js 环境，可访问文件系统、系统托盘 — 渲染进程 (renderer/) ← Chromium，显示网页内容 — app.js ← Three.js 3D 场景 — editor.html ← Pose 编辑器

桌面窗口特性

参数	值	效果
尺寸	380 × 640	竖屏，类似手机比例
transparent	true	窗口背景透明
frame	false	无标题栏、无边框
alwaysOnTop	true	始终显示在最前面
resizable	false	固定大小

效果：一个透明无边框、始终置顶的 3D 角色窗口，悬浮在桌面右侧，不挡其他窗口，像桌宠一样。

依赖管理（Node.js / npm）

electron

three.js r167

@pixiv/three-vrm 2.1.2

Stable Diffusion 是什么？

Stable Diffusion (SD) 是一个开源的 AI 图像生成模型，输入文字描述，输出图像。雪 Bot 用它生成角色图像发送给用户。

LoRA — 教 SD "认识" 一个特定角色

SD 默认不知道"雪"长什么样。LoRA (Low-Rank Adaptation) 是一种在不修改原始模型的情况下，让模型学会新知识的微调技术：

- 1 准备训练数据：**用 VRM 模型渲染 11 张不同角度的图片，每张配一个描述标签 (caption)
- 2 训练：**让 SD 模型对比"正确图像"和"它生成的图像"的差距，调整一小部分参数
- 3 输出：**一个 ~37MB 的 `.safetensors` 文件，存储了角色外观的"特征偏移量"
- 4 使用：**生图时加上 `<lora:yuki_lora_v1:0.8>`，SD 就会把这些特征叠加到生成结果上

为什么从 VRM 渲染训练数据？ VRM 是 3D 模型，可以随时生成任意角度、任意姿势的图，不需要手动画参考图。而且外观完全一致，不存在风格差异。

训练流水线 (lora/ 目录)

```
lora/1_setup.ps1 ← 首次运行，克隆 kohya-ss/sd-scripts | lora/gen_data.py ← 调 vrm_render.py, 渲染 11 张训练图 | 输出到 data/yuki_lora/img/20_yukixue/ | 自动生成对应的 caption .txt 文件 | sd-scripts/train_network.py ← kohya 训练脚本 | 底模: Realistic Vision V5.1 fp16 | 30 epochs, batch_size=2, network_dim=32 | GPU: RTX 4070 12GB, fp16 精度 ▼  
data/lora/output/yuki_lora_v1.safetensors ← 成品
```

关键训练参数解释

参数	值	含义
network_dim	32	LoRA 的"容量"，越大学得越多也越容易过拟合
network_alpha	16	学习强度缩放，通常是 dim 的一半
num_repeats	20	每张图重复 20 次，弥补训练数据少的问题
clip_skip	2	跳过最后 1 层文字编码，Realistic Vision 的标准设置

mixed_precision	fp16	半精度浮点数，节省 VRAM，速度更快
cache_latents	true	缓存图片的潜空间编码，不用每步重算

GPT-SoVITS 是什么？

GPT-SoVITS 是一个开源的语音合成系统，只需要几秒钟的参考音频，就能克隆对应的音色，生成任意文字的语音。

工作流程

`handle_reply.py` 生成回复文字 | ▼ `send_voice.py` | POST `http://127.0.0.1:9880/tts` | { | `text`: "要合成的文字", | `ref_audio`: "assets/voice_ref/任命助理.wav", | `ref_text`: "参考音频对应的文字" | } ▼
GPT-SoVITS API (`start_voice_api.bat` 启动) | 加载模型权重 | → GPT 模型 (`42-e15.ckpt`): 文字→音素序列
| → SoVITS 模型 (`42_e8_s152.pth`): 音素→音频波形 ▼ 返回 `.wav` 音频文件 | ▼ Telegram `sendVoice` → 发送语音消息

模型文件

文件	大小	作用
<code>42-e15.ckpt</code>	~400MB	GPT 模型，将文字转为音素序列（决定节奏、停顿）
<code>42_e8_s152.pth</code>	~100MB	SoVITS 模型，将音素转为音频波形（决定音色）
<code>任命助理.wav</code>	几秒钟	参考音频，用来确定要克隆的音色

注意：语音功能是可选的。如果不启动 GPT-SoVITS（`scripts/start_voice_api.bat`），Bot 会静默跳过语音合成步骤，只发文字消息。

完整技术栈

层次	技术	版本	用途
AI / LLM	Claude	sonnet-4-6	主要对话与决策 AI
	BAAI/bge-small-zh-v1.5	—	记忆向量化（本地）
	sentence-transformers	—	向量计算库
图像生成	Stable Diffusion 1.5	—	底模
	kohya-ss/sd-scripts	—	LoRA 训练框架
3D 渲染	Three.js	r167	WebGL 3D 渲染
	@pixiv/three-vrm	2.1.2	VRM 模型解析
	Playwright	1.60	无头浏览器渲染
数据存储	SQLite	—	记忆数据库
	JSON 文件	—	状态信号、配置
后端服务	FastAPI	—	Web 管理面板 API
	Uvicorn	—	ASGI 服务器
桌面应用	Electron	—	透明置顶桌面窗口
消息平台	Telegram Bot API	—	与用户通信
语音合成	GPT-SoVITS	—	文字转角色语音
运行环境	Python 3.10	Conda (sd env)	后端运行环境
	Node.js / npm	—	桌面应用运行环境
GPU	NVIDIA RTX 4070	12GB VRAM	LoRA 训练、SD 生图

完整数据流图

用户侧 ———— 薰 (Telegram 手机) | ▼ 消息 `tg_daemon.py` —写→
`pending_reply.json` | ▼ 启动 `handle_reply.py` —读→ `memory.db` (历史 + 相关记忆) —调→

Claude API (生成回复) | 调 → GPT-SoVITS :9880 (生成语音) | 发 → Telegram API (发送) | 写
→ memory.db (存对话 + 生活日志) | 自主侧 | lt_daemon.py | 写 →
pending_tick.json | ▼ 检测到文件 claude_monitor.py | 执行 → claude -p lifetick_prompt.txt |
读 memory.db + character.txt | AI 决策: 是否发消息 | 写 next_tick.json | 管理侧
浏览器 :8765 | → memory.db (记忆管理) | → Yuki.vrm (3D 查看) |
训练侧 (离线) | → vrm_render.py → Playwright → 11 张训练图 | → sd-scripts →
yuki_lora_v1.safetensors | → SD WebUI (生成角色图像)

端口占用一览

端口	服务	启动方式
8765	记忆管理面板 (FastAPI)	memory/admin/server.py
9880	GPT-SoVITS 语音 API	scripts/start_voice_api.bat

学习路线建议：如果你想理解这个项目，建议按以下顺序学习：① Python 基础 → ② SQLite / 数据库 → ③ HTTP API (FastAPI) → ④ Telegram Bot API → ⑤ LLM API 调用 → ⑥ Three.js 3D 入门 → ⑦ Stable Diffusion 原理